

SemEval-2020 Task 7: Assessing Humor in Edited News Headlines - Subtask 1 (Regression)

Matthew Vu, Vu Doan, Prapurna Penmathsa and Amrisha Dwivedi
George Mason University

Abstract

This paper delves into the task of humor detection in edited news headlines with a focus on predicting their funniness rating. In our research, we were able to experiment with and train four ALBERT (A-LITE BERT) models with different configurations to predict these funniness ratings. For training, validation, and testing we used a large randomly partitioned corpus consisting of edited news headlines and their funniness ratings (based on the judgement of human annotators). Despite its simplicity, our best and final ALBERT model configuration achieved competitive results compared to more complex architectures because it attained an RMSE of 0.5410 against the validation set and an RMSE of 0.5510 against the test set. This best configuration used the Adam optimizer and a constant decay factor of $1e-8$ to exponentially decrease the learning rate for every subsequent batch during the training process.

1 Introduction

The task of humor detection in text, especially within edited news headlines, presents a multifaceted challenge in natural language processing (NLP). Humor detection requires a deep understanding of linguistic nuances and contextual cues that contribute to the perception of funniness. Predicting the level of humor in these headlines is crucial for various applications, such as enhancing chatbots, virtual assistants, and content personalization. Moreover, the intricacies of humor lie in its subjective nature, often influenced by individual interpretations and societal contexts. The research focus on predicting funniness ratings in edited headlines serves as a fundamental step in advancing NLP tools, enabling machines to comprehend and generate humor effectively. This analysis task not only illuminates the mechanisms behind humor creation but also facilitates the development of computational models capable of discerning and generating humor in textual content, thus fostering

more engaging and contextually relevant communication.

In this project, we aim to explore and assess humor detection in edited news headlines by experimenting with four distinct configurations of ALBERT (A-Lite BERT) models. ALBERT models, known for their parameter-efficient design compared to BERT, offer a promising avenue for understanding the funniness rating in textual content. Each ALBERT model will undergo varied configurations to predict the level of humor in edited news headlines, contributing to the advancement of natural language processing tools specialized in humor detection. Through this experimentation, we aim to uncover the most effective ALBERT model configuration for predicting funniness ratings, facilitating the development of robust computational models for humor analysis in textual data.

2 Past Research

Jin et al. (2020) underscore the utilization of transformer-based models for humor detection in edited news headlines, shedding light on the intricate nature of humor and advocating for future research that incorporates additional elements beyond incongruity. The researchers provide several significant insights, including the use of pre-trained transformer-based models, such as RoBERTa, to evaluate the humor in edited news headlines. It highlights the effectiveness of their contrastive approach, which involves comparing the original and edited headlines to gauge humor. Additionally, the paper explores a non-contrastive approach that performs similarly, implying an implicit understanding of contrast. Notably, it observes that model judgments exhibit weak correlations with human judgments but strong correlations with each other. The research emphasizes that while incongruity is a common feature in humor, it is neither a sufficient nor a necessary condition for humor. The researchers acknowledge that humor can have diverse

origins, encompassing elements such as sarcasm, puns, and cultural commentary. These findings provide valuable insights into the multifaceted nature of humor detection in natural language processing.

Khwaileh and Al-As'ad (2020) use of different pre-trained word embeddings, with a focus on ELMo and BERT. ELMo is a contextualized deep model that assigns word vectors based on context. BERT, on the other hand, uses a transformer method to convert words into embeddings, providing a deep bidirectional understanding of the text. The researchers provide an overview of these models and their applications. The performance of the models is measured using Root Mean Square Error (RMSE), which is calculated based on Mean Square Error (MSE) by comparing observed and predicted values. The authors' approach emphasizes preprocessing with a focus on humor sense detection, utilizing embeddings from the ELMo language model, and ensembling the results obtained using a machine learning model, specifically Naïve Bayes (NB) combined with deep learning pre-trained models. Their approach, ELMo-NB, achieved a score of 0.5642 on the competition leaderboard, with results measured by Root Mean Squared Error (RMSE).

Meaney et al. (2020) chose ERNIE 2.0 over BERT due to ERNIE's ability to capture more lexical, syntactic, and semantic information, including the relationship between entities. ERNIE's training on diverse domains, including encyclopedias and news articles, is particularly advantageous in datasets like Funlines, which are based on news headlines with numerous named entities. In terms of model configuration, ERNIE base with specific hyperparameters is used for transfer learning. Fine-tuning is applied to both regression and classification tasks, resulting in improved performance, especially in the classification task. The researchers acknowledged the importance of hyperparameter optimization in fine-tuning layers. While transfer learning models have excelled in various NLP tasks, their performance in humor detection highlights the significance of hyperparameter tuning.

Tuncer (2021) chose to utilize ALBERT (A Lite BERT) transformer models to predict the mean funniness score in edited news headlines. The reason he decided to leverage an ALBERT transformer model over the traditional BERT model was ALBERT has fewer parameters compared to BERT. This in turn allows the model to be trained faster.

In his research, he used the datasets provided for SemEval-2020 Task 7: Assessing Humor in Edited News Headlines - Subtask 1 in order to train, validate, and experiment with two ALBERT models. One model used the RMSprop optimizer and the other used the Adam optimizer. Nonetheless, for both models, he decreased the learning rate by a constant value during the training process. In the end, his two ALBERT models using either the RMSprop or Adam optimizers were able to achieve RMSE values of 0.4569 and 0.4944 against the validation set, respectively. Thus, he deemed the RMSprop optimizer to be the better optimizer for ALBERT for mean funniness score predictions. Additionally, it is important to note that we based and built our own methods off of Tuncer's research, methods, and code.

3 Data

The dataset called "Humicroedit" is a valuable resource for computational humor research. This dataset is designed to facilitate the investigation of how humor is generated in text, specifically in the context of news headlines. It consists of original English news headlines and versions of those headlines that have undergone small word replacement edits to make them funny (Hossain et al., 2020). The primary objective is to gain insights into the mechanics of humor and what makes headlines humorous. The data collection process involves gathering news headlines from popular subreddits on Reddit, filtering them for quality, and then annotating them with the help of qualified human annotators.

The dataset comprises a total of 15,095 edited humorous headlines, each of which has been evaluated by five distinct individuals, including crowdsourcing editors and judges. For each edited headline, five humor scores have been assigned, indicating the level of humor perceived by the judges. These humor scores fall within the range of 0 to 3, with 0 indicating that the headline is not funny, and 3 suggesting that it is particularly humorous based on the integer scale developed by Hossain et al. (2020). See Table 1. The overall mean funniness score across all edited headlines in the dataset is 0.94, indicating a notable degree of humor present in these headlines. The headlines are concise and vary in length, from a minimum of four words to a maximum of 20 words, allowing researchers to explore humor generation and recognition across

different textual structures.

Number	Funniness
0	Not Funny
1	Slightly Funny
2	Moderately Funny
3	Funny

Table 1: Funniness categories

To create the "Humicroedit" dataset, expert annotators are recruited through Amazon Mechanical Turk and qualified for two main tasks: editing headlines to inject humor and judging the humor of these edited headlines. Humor judges are selected based on their ability to objectively assess the funniness of edited headlines, while humor editors are chosen for their proficiency in making headlines humorous through micro-edits. The dataset's quality is maintained through careful data control measures, including the correction of spelling mistakes, removal of concatenated words, and the introduction of diversity in annotations. Editors and judges whose performance deviates from the norm are removed to ensure data quality. The data is stored as shown in Figure 1. Where original is the unedited news headline, edit is the word that replaced an input word in the original headline, grades is the annotator score and meanGrade is the mean funniness grade.

The dataset offers a rich opportunity for various computational humor tasks, such as understanding the mechanisms that render headlines funny, predicting humor, ranking edits on a funniness scale, generating humorous headlines, and personalizing funny headlines for readers. The headlines in the dataset are not restricted by specific templates, and they convey substantial information with minimal words. Additionally, achieving humor in headlines involves deep semantic understanding and reasoning, including world knowledge and common sense. Overall, the "Humicroedit" dataset presents a valuable foundation for exploring the nuances of humor in text and advancing the development of NLP tools for humor generation and recognition in topical contexts.

For SemEval-2020 Task 7: Assessing Humor in Edited News Headlines - Subtask 1, the Humicroedit dataset is provided to participants in three files (or dataset subsets): "train.csv", "dev.csv", and "test.csv". Essentially, the Humicroedit dataset is randomly partitioned into three parts. The first par-

tition is the training set and is saved as "train.csv"; the second partition is the validation set and is saved as "dev.csv"; and the third partition is the test set and is saved as "test.csv". The training, validation, and test sets consist of 9652 (64%), 2419 (16%), and 3024 (20%) instances from the Humicroedit dataset (Hossain et al., 2020).

id	original	edit	grades	mean Grade
14530	France is ' hunting down its citizens who joined <Isis/> ' without trial in Iraq	twins	10000	0.2
13034	Pentagon claims 2,000 % increase in Russian trolls after <Syria/> strikes . What does that mean ?	bowling	33110	1.6
8731	Iceland PM Calls Snap Vote as Pedophile Furor Crashes <Coalition/>	party	22100	1.0
76	In an apparent first , Iran and Israel <engage/> each other militarily	slap	20000	0.4
6164	Trump was told weeks ago that Flynn misled <Vice/> President .	school	0	0.0

Figure 1: Data example for Task 1

4 Methods

In order to accomplish the first subtask for SemEval-2020 Task 7, we experimented with four 'TFAIbertForSequenceClassification' (ALBERT transformer-based) models with different configurations to predict the numerical mean funniness scores of edited news headlines. Nonetheless, our methods were heavily based on the previous research and code conducted and written by Tuncer (2021) to leverage ALBERT transformer models to predict these mean funniness scores.

Essentially, the ALBERT (A-Lite BERT) transformer model is a modified BERT model that uses parameter-reduction techniques, mainly factorized embedding parameterization and cross-parameter sharing, to significantly reduce the number of parameters and computational cost to train. Factorized embedding parameterization allows ALBERT to use "separate embedding matrices for different types of tokens (e.g. words, subwords, positional encodings), instead of using a single embedding matrix for all input tokens". Additionally, cross-layer parameter sharing allows ALBERT to use "parameter sharing between the transformer layers" (Lan et al., 2020; udit, 2023; Tuncer, 2021).

The first pair of ALBERT models (nicknamed models 1 and 2) that we experimented on were con-

figured to use the same exponential decay learning rate scheduler to directly decrease the learning rate every 603 batches over the training process, but one of these models used the RMSprop optimizer and the other used the Adam optimizer. Additionally, the hyperparameters set for the learning rate scheduler were an initial learning rate of $1e-5$, decay steps of 603, decay rate of 0.9, and number of epochs of 3. These hyperparameters were set to allow the exponential decay learning rate scheduler to start with an initial learning rate of $1e-5$ with the learning rate decreasing every 603 batches by a factor of 0.90 (Tensorflow, 2023). The example code to initialize the exponential decay learning rate scheduler is shown in Figure 2. The example code to fit ALBERT model 1 using the scheduler and RMSprop optimizer is shown in Figure 3. The example code to fit ALBERT model 2 using the scheduler and Adam optimizer is shown in Figures 4.

```
# Define initial learning rate
initial_learning_rate = 1e-5

# Define decay steps and rate
decay_steps = 603
decay_rate = 0.9

# Create learning rate schedule
lr_schedule = ExponentialDecay(
    ... initial_learning_rate,
    ... decay_steps=decay_steps,
    ... decay_rate=decay_rate)
...
```

Figure 2: The code to initialize the exponential decay learning rate scheduler for models 1 and 2

On the other hand, the second pair of ALBERT models (nicknamed models 3 and 4) were configured to use a constant decay factor of $1e-8$ to exponentially decrease the learning rate every subsequent batch during the training process, but one of these models used the RMSprop optimizer and the other used the Adam optimizer (hans, 2018; Tuncer, 2021). Essentially models 3 and 4 had three hyperparameters: an initial learning rate of $1e-5$, decay factor of $1e-8$, and the number of epochs equal to 3. The example code to fit ALBERT models 3 and 4 with the constant decay factor of $1e-8$ to exponentially decrease the learning rate every subsequent batch using either the RMSprop or Adam optimizers are shown in Figures 5 and 6.

The important commonalities between all four ALBERT models were they were trained with 3 epochs, an initial learning rate of $1e-5$, and the

training set provided for the shared task.

The technique of using an exponential decay learning rate scheduler to directly change the learning rate by a certain proportion every 603 batches over the training process, found in the first pair of models (1 and 2), was unique to our work and not performed in the research conducted by Tuncer (2021). However, he **did** use the same exact code and methods as we did to create the second pair of models (3 and 4) that used a constant decay factor of $1e-8$ to exponentially decrease the learning rate every subsequent batch during the training process.

Now, in order to determine the predictive performance of the models, the root mean square error (RMSE) evaluation metric was used. Also, as part of the model evaluation process, we first computed two maximum RMSE baselines.

According to Hossain (2019) and Hossain et al. (2020), who influenced these maximum baseline methods, one baseline RMSE value was equal to the validation set's ("dev.csv") RMSE when the predicted mean funniness score of all edited news headlines in that validation set were equal to the average mean funniness score of all edited news headlines in the training set ("train.csv"). This was referred to as the validation set baseline RMSE. The other baseline RMSE value was equal to the test set's ("test.csv") RMSE when the predicted funniness score of all edited news headlines in the test set were equal to the average mean funniness score of all edited news headlines in the training set ("train.csv"). This was referred to as the test set baseline RMSE. Any model that had a higher validation or test set RMSE compared to the maximum baselines was deemed unsuccessful because we might as well just assign every prediction to be equal to the average mean funniness score from the training set rather than training, fitting, and using the model.

Now, the best performing and final model was chosen based on the following criteria: the model's computed RMSE against the validation set must be lower than the validation set baseline RMSE, and the model's validation set RMSE must be the lowest compared to other models.

Once the best performing and final model was chosen, its generalization error was found by computing the model's RMSE value against the test set. This told us how well the model ought to perform on data not seen during the model training and selection process. Furthermore, this entire project


```

# first train the model using the RMSprop optimizer
# create an albert model specialised on regression
model_rmsprop = NN().get_nn()

# use the RMSprop optimizer
optimizer_rmsprop = RMSprop(learning_rate = lr_schedule)

# compile the model with the appropriate optimizer, loss function, and evaluation metric
model_rmsprop.compile(optimizer=optimizer_rmsprop, loss=MeanSquaredError(),
...|...|...metrics=[RootMeanSquaredError('root_mean_squared_error')])

print("Fit model with RMSprop optimizer on training data.")

# train the model on the training set for 3 epochs
# and using the validation set for validation
history_rmsprop = model_rmsprop.fit(train_dataset, validation_data=dev_dataset, epochs=3)
print("RMSprop model successfully trained")

```

Figure 3: The code to fit ALBERT model 1 using the exponential decay learning rate scheduler and RMSprop optimizer

```

# create an albert model specialised on regression
model_adam = NN().get_nn()

# use the Adam optimizer
optimizer_adam = Adam(learning_rate = lr_schedule)

# compile the model with the appropriate optimizer, loss function, and evaluation metric
model_adam.compile(optimizer=optimizer_adam, loss=MeanSquaredError(),
...|...|...metrics=[RootMeanSquaredError('root_mean_squared_error')])

# train the model on the training set for 3 epochs
# and using the validation set for validation
history_adam = model_adam.fit(train_dataset, validation_data=dev_dataset, epochs=3)
print("Adam model successfully trained")

```

Figure 4: The code to fit ALBERT model 2 using the exponential decay learning rate scheduler and Adam optimizer

was deemed a success if the final model had a test set RMSE value lower than the test set’s baseline RMSE. Now, it is important to note that unlike [Tuncer \(2021\)](#), whose research we based our methods on, he did not compute the generalization error on the test set and, instead, opted to simply predict the mean funniness scores of five random samples in the test set.

In general, the training set was used to train the models; the validation set was used to assess the predictive performance of the candidate models on unseen data, tune hyperparameters, and choose the best model; and the test set was used to compute the generalization error in order to assess how well the model would perform on data not seen in the project’s lifecycle.

Now, in terms of data processing to get the data in a format that the ALBERT models could train on was rather complicated. However, in summary and simply put, in order to process the data, the microedits were replaced in the actual news headlines to turn them into their humorous versions, the words and punctuation were tokenized, an attention mask was created, and labels (mean funniness scores) were extracted and placed in their own vector.

Furthermore, in order to conduct and document the results of these methods described for our project, we utilized Jupyter Notebooks and Python in addition to Python’s several NLP and data science libraries. For instance, we made extensive use of Pandas, Numpy, and Tensorflow.

5 Results and Discussion

As one can see from Table 2, the maximum validation and test set baseline RMSE values are 0.5784 and 0.5747, respectively. This meant that a model could only be deemed successful and possibly chosen to be the final model if its validation set RMSE was below 0.5784. Also, in order for this project to be deemed successful, the finally chosen best model must have a test set RMSE value below 0.5747.

Dataset	Baseline RMSE
Validation set	0.5784
Test set	0.5747

Table 2: The maximum validation and test set RMSE baselines

As shown in Table 3, all four ALBERT mod-

```

# create an albert model specialised on regression
model_rmsprop_original = NN().get_nn()

# use the RMSprop optimizer
optimizer_rmsprop_original = tf.keras.optimizers.legacy.RMSprop(learning_rate = 1e-5, decay = 1e-8)

# compile the model with the appropriate optimizer, loss function, and evaluation metric
model_rmsprop_original.compile(optimizer=optimizer_rmsprop_original, loss=MeanSquaredError(),
                               metrics=[RootMeanSquaredError('root_mean_squared_error')])

print("Fit model on training data.")
# train the model on the training set for 3 epochs
# using the validation set for validation
history_rmsprop_original = model_rmsprop_original.fit(train_dataset, validation_data=dev_dataset, epochs=3)
print("Original RMSprop model successfully trained")

```

Figure 5: The code to fit ALBERT model 3 with the constant decay factor of $1e-8$ to exponentially decrease the learning rate every subsequent batch and RMSprop optimizer

```

# create an albert model specialised on regression
model_adam_original = NN().get_nn()

# use the Adam optimizer
optimizer_adam_original = tf.keras.optimizers.legacy.Adam(learning_rate = 1e-5, decay = 1e-8)

# compile the model with the appropriate optimizer, loss function, and evaluation metric
model_adam_original.compile(optimizer=optimizer_adam_original, loss=MeanSquaredError(),
                             metrics=[RootMeanSquaredError('root_mean_squared_error')])

print("Fit model on training data.")
# train the model on the training set for 3 epochs
# using the validation set for validation
history_adam_original = model_adam_original.fit(train_dataset, validation_data=dev_dataset, epochs=3)
print("Adam model successfully trained")

```

Figure 6: The code to fit ALBERT model 4 with the constant decay factor of $1e-8$ to exponentially decrease the learning rate every subsequent batch and Adam optimizer

els that we experimented on performed better than the validation set baseline RMSE and, therefore, were all candidates to be chosen as the final model. Nonetheless, model 4 that used a constant decay factor of $1e-8$ to exponentially decrease the learning rate every subsequent batch during the training process and the Adam optimizer performed the best against the validation set with a validation set RMSE value of 0.5410. Because model 4 performed the best compared to the others, it was chosen as the final and best model for future predictions and deployment.

Also, model 2 that used the exponential decay learning rate scheduler to directly decrease the learning rate every 603 batches over the training process and the Adam optimizer came close in second place with a validation set RMSE of 0.5457. On the other hand, models 1 and 3 that used the RMSprop optimizer performed worse than the models that used the Adam optimizer. So, in our experiments, the models using the Adam optimizer were better at predicting mean funniness scores than the ones using the RMSprop optimizer.

However, as shown in Table 3 and Table 4, models 3 and 4 that were trained using a constant decay factor of $1e-8$ to exponentially decrease the

learning rate every subsequent batch during the training process performed differently on the validation set in our own experiments compared to the experiments conducted by Tuncer (2021). In our own experiments, models 3 (that used the RMSprop optimizer) and model 4 (that used the Adam optimizer) we were able to achieve validation set RMSE values of 0.5515 and 0.5410, respectively. On the other hand, Tuncer was able to achieve validation set RMSE values of 0.4569 and 0.4944 using either the RMSprop or Adam optimizers, respectively. This was very strange because these models that we experimented on were exactly the same as Tuncer’s. The discrepancies in validation set RMSE values could be due to a number of reasons. The first possible reason could be that different versions of Tensorflow were used because Tuncer used version 2.6 and we used version 2.14. The second possible reason could be that we used different random seeds. In fact, Tuncer did not set a random seed.

Finally, the best model from our project (model 4) had a generalization error and RMSE value against the test set of 0.5510. This indicated that future mean funniness score predictions using model 4 would differ from their true values by 0.5510.

Model Number	Learning Rate Decay Method	Hyperparameters	Optimizer	Validation Set RMSE
1	Exponential Decay Learning Rate Scheduler to Decrease Learning Rate Every 603 Batches	Initial learning rate = $1e-5$, Decay steps = 603, Decay rate = 0.9, Epochs = 3	RMSprop	0.5498
2	Exponential Decay Learning Rate Scheduler to Decrease Learning Rate Every 603 Batches	Initial learning rate = $1e-5$, Decay steps = 603, Decay rate = 0.9, Epochs = 3	Adam	0.5457
3	Constant decay factor to exponentially decrease learning rate every subsequent batch	Initial learning rate = $1e-5$, Decay factor = $1e-8$, Epochs = 3	RMSprop	0.5515
4	Constant decay factor to exponentially decrease learning rate every subsequent batch	Initial learning rate = $1e-5$, Decay factor = $1e-8$, Epochs = 3	Adam	0.5410

Table 3: The four ALBERT models (1, 2, 3, and 4) used and their corresponding RMSE against the validation set from our own experiments

Model Number	Learning Rate Decay Method	Hyperparameters	Optimizer	Validation Set RMSE
3	Constant decay factor to exponentially decrease learning rate every subsequent batch	Initial learning rate = $1e-5$, Decay factor = $1e-8$, Epochs = 3	RMSprop	0.4569
4	Constant decay factor to exponentially decrease learning rate every subsequent batch	Initial Learning Rate = $1e-5$, Decay factor = $1e-8$, Epochs = 3	Adam	0.4944

Table 4: The two ALBERT models (3 and 4) used and their corresponding RMSE against the validation set from (Tuncer, 2021)

Also, model 4 generalized well, and the project was a success because the model’s test set RMSE was lower than the test set baseline RMSE of 0.5747.

6 Conclusion

In conclusion, we experimented with four ALBERT models with different configurations to predict mean funniness scores of edited news headlines. Our first pair of models were configured to use the same exponential decay learning rate scheduler to directly decrease the learning rate every 603 batches over the training process, but one of these models used the RMSprop optimizer and the other used the Adam optimizer. The second pair of models were configured to use a constant decay factor of $1e-8$ to exponentially decrease the learning rate

every subsequent batch during the training process, but one of these models used the RMSprop optimizer and the other used the Adam optimizer. In the end, our best and final model was configured to use a constant decay factor of $1e-8$ to exponentially decrease the learning rate every subsequent batch during the training process and the Adam optimizer with a validation set RMSE value of 0.5410 and a test set RMSE value of 0.5510.

References

- hans. 2018. [Difference between rmsprop with momentum and adam optimizers](#). *StackOverFlow*.
- Nabil Hossain. 2019. [semeval-2020-task-7-humicroedit](#). *GitHub*.

- Nabil Hossain, John Krumm, Michael Gamon, and Henry Kautz. 2020. [Semeval-2020 task 7: Assessing humor in edited news headlines](#). In *Proceedings of the 14th International Workshop on Semantic Evaluation*, pages 746–758, Barcelona, Spain (Online).
- Shuning Jin, Yue Yin, XianE Tang, and Ted Pedersen. 2020. [Duluth at semeval-2020 task 7: Using surprise as a key to unlock humorous headlines](#). In *Proceedings of the 14th International Workshop on Semantic Evaluation*, pages 986–994, Barcelona, Spain (Online).
- Enas Khwaileh and Muntaha A. Al-As’ad. 2020. [Elmo-nb at semeval-2020 task 7: Assessing sense of humor in editednews headlines using elmo and nb](#). In *Proceedings of the 14th International Workshop on Semantic Evaluation*, pages 1001–1007, Barcelona, Spain (Online).
- Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. 2020. [Albert: A lite bert for self-supervised learning of language representations](#). In *the International Conference on Learning Representations*.
- J. A. Meaney, Steven Wilson, and Walid Magdy. 2020. [Smash at semeval-2020 task 7: Optimizing the hyperparameters of ernie 2.0 for humor ranking and rating](#). In *Proceedings of the 14th International Workshop on Semantic Evaluation*, pages 1049–1054, Barcelona, Spain (Online).
- Tensorflow. 2023. [tf.keras.optimizers.schedules.exponentialdecay](#). *TensorFlow v2.14.0*.
- Salih Tuncer. 2021. [Semeval-2020 task 7: Assessing humor in edited news headlines - subtask 1 \(regression\)](#). *GitHub*.
- udit. 2023. [Albert: Achieving state-of-the-art results with a compact and efficient transformer-based model for natural language processing](#). *Medium*.